LION GPU LAB

GPU 服务器学生使用手册

通用版 · 独立服务器排队体系

所有 GPU 程序必须通过 Slurm 运行

版本 v2.0 | 2026-08-16

适用于实验室所有已接入 Slurm 的 GPU 服务器

> **核心规则：SSH 只用于编辑代码、传输文件、查看状态和提交任务；所有 GPU 程序必须通过 Slurm 运行。**

1. 先理解服务器的组织方式

实验室的 GPU 服务器采用“每台服务器独立排队”的方式：

- 每台服务器有自己的 Slurm 控制器、任务队列和本地磁盘。
- 学生登录哪台服务器，就在那台服务器上提交任务。
- GPU 配额和同时运行上限按每台服务器分别计算，不做跨服务器合并。
- 服务器之间没有共享硬盘。切换服务器前，需要自行同步代码、数据、环境和 checkpoint。
- 统一监控网站可以汇总查看各服务器，但网站只读，不能代替 `sbatch` 提交任务。

服务器地址、开放范围和监控网站连接方式由管理员另行发布。

2. 三分钟上手

2.1 登录后先看状态

```
lion-status
nvidia-smi
sinfo
squeue -u "$USER"
```

说明：

- `lion-status`：查看本机健康状态、队列和告警摘要。
- `nvidia-smi`：SSH 下显示每 2 秒更新的只读快照，不会授予真实 GPU 访问权限。
- `sinfo`：查看节点和分区状态。
- `squeue -u "$USER"`：只查看自己的运行和排队任务。

2.2 提交第一个单卡任务

新建 `train.slurm`：

```
#!/bin/bash
#SBATCH --job-name=train
#SBATCH --partition=normal
#SBATCH --gres=gpu:1
#SBATCH --time=1-00:00:00
#SBATCH --output=slurm-%x-%j.out
#SBATCH --error=slurm-%x-%j.err

set -euo pipefail

source /path/to/conda.sh
conda activate myenv
python train.py
```

提交和查看：

```
sbatch train.slurm
squeue -u "$USER"
```

2.3 必须遵守的提交约定

- 必须使用 `--gres=gpu:N` 明确申请 GPU；不申请 GPU 的学生任务会被拒绝。
- 只选择 `normal`、`debug`、`idle` 或 `reserved` 分区。
- 不要填写 `--qos`；系统会根据分区和硕士/博士角色自动选择 QoS。
- 未写 `--partition` 时，默认使用 `normal`。
- 未写 CPU 和内存时，每张 GPU 默认分配 8 个逻辑 CPU 和 32 GiB 内存。
- 不要指定物理 GPU 编号。Slurm 会设置 `CUDA_VISIBLE_DEVICES`，程序只使用作业内可见的 GPU。

3. 正式使用规则

3.1 分区规则

分区	用途	GPU 上限	默认/最长时间	是否可能被抢占
`normal`	正式训练与稳定实验	硕士 2；博士 4	24 小时 / 72 小时	否
`debug`	交互调试、环境和显存测试	每人合计最多 2	30 分钟 / 30 分钟	否
`idle`	利用暂时空闲的 GPU	每人最多 8	24 小时 / 72 小时	是，会重新排队

分区	用途	GPU 上限	默认/最长时间	是否可能被抢占
`reserved`	管理员批准的特殊预约	按预约，最高 8	24 小时 / 7 天	预约窗口内优先

3.2 硕士与博士

角色	`normal` GPU 上限	QoS 优先级	FairShare 权重
硕士 `master`	每台服务器最多同时 2 张	100	2
博士 `phd`	每台服务器最多同时 4 张	150	3

角色说明：

- 新批准用户默认是硕士 `master`；博士由管理员核实后改为 `phd`。
- 博士约有 1.5 倍软优先和 FairShare 权重，但不会中断已经运行的硕士 `normal` 作业。
- `debug` 对所有人统一最多 2 张 GPU；博士的 4 卡权益用于 `normal`。
- `normal` 与 `debug` 共用角色的 GPU 同时运行上限。

3.3 排队、公平份额和提交数量

- `normal/debug`、`idle`、`reserved` 每类最多保留 30 个已提交任务（含运行和排队）。
- 排队不是简单的先到先得。系统综合任务类型、角色、近期 GPU 用量和等待时间排序。
- 公平份额历史半衰期为 7 天；近期使用较多的人，下一轮优先级会逐步下降，之后自动恢复。
- `idle` 的 GPU 时间按 10% 计入公平份额，但不代表只能占用 10% 的 GPU。
- 没有资源竞争时，低优先级任务仍可立即运行。
- 反复取消再提交不会清除历史用量，也不会提高优先级。

4. 选择正确的分区

4.1 normal：正式训练

`normal` 适合已经完成调试、能够稳定运行的实验。

硕士或博士双卡示例：

```
#!/bin/bash
#SBATCH --job-name=train-2gpu
#SBATCH --partition=normal
#SBATCH --gres=gpu:2
#SBATCH --time=2-00:00:00
#SBATCH --cpus-per-task=16
#SBATCH --mem=64G
#SBATCH --output=slurm-%x-%j.out
#SBATCH --error=slurm-%x-%j.err

set -euo pipefail
source /path/to/conda.sh
conda activate myenv
python -m torch.distributed.run --nproc_per_node=2 train.py
```

博士四卡只需改为：

```
#SBATCH --gres=gpu:4
#SBATCH --cpus-per-task=32
#SBATCH --mem=128G
```

多张 GPU 不会自动合并显存。程序必须支持数据并行、模型并行或张量并行。

4.2 debug：短时间交互调试

单卡调试：

```
srun --partition=debug --gres=gpu:1 --time=00:30:00 --pty bash
```

双卡调试：

```
srun --partition=debug --gres=gpu:2 --time=00:30:00 --pty bash
```

进入作业后可以运行真实的 `nvidia-smi`、Python 或 CUDA 程序；结束时执行 `exit`。`debug` 不能用于长期训练。

4.3 idle：使用空闲卡

`idle` 适合可随时中断并从 checkpoint 恢复的实验。

```
#!/bin/bash
#SBATCH --job-name=idle-train
#SBATCH --partition=idle
#SBATCH --gres=gpu:4
#SBATCH --time=3-00:00:00
#SBATCH --requeue
#SBATCH --signal=B:USR1@300
#SBATCH --output=slurm-%x-%j.out
#SBATCH --error=slurm-%x-%j.err

set -euo pipefail
source /path/to/conda.sh
conda activate myenv
python train_with_checkpoint.py
```

注意：

- 程序必须定期保存 checkpoint，并确认能够自动或手动恢复。
- `--signal=B:USR1@300` 只保证到达时间上限前 5 分钟发信号，不保证抢占前预警。
- `idle` 最长可写 72 小时，但可能运行几分钟后就被重新排队。
- 不能安全恢复的任务应使用 `normal`。

4.4 reserved：特殊预约

预约由管理员创建。提交前管理员会提供预约名、GPU 数量、开始时间和持续时间。

```
#!/bin/bash
#SBATCH --job-name=paper-run
#SBATCH --partition=reserved
#SBATCH --reservation=REPLACE_WITH_RESERVATION_NAME
#SBATCH --gres=gpu:2
#SBATCH --time=2-00:00:00
#SBATCH --output=slurm-%x-%j.out
#SBATCH --error=slurm-%x-%j.err

set -euo pipefail
source /path/to/conda.sh
conda activate myenv
python train.py
```

- 不要写 `--qos=reserved`，系统会自动选择。
- 任务时长必须能放进预约剩余时间。
- 没有预约名的 `reserved` 任务不会运行。
- 预约最长 7 天，具体资源以批准结果为准。

5. CPU 和内存

CPU、内存不设“每人固定额度”，但每个任务只能使用自己申报的资源。

GPU 数	默认 CPU	默认内存
1	8	32 GiB
2	16	64 GiB
4	32	128 GiB
8	64	256 GiB

有真实需要时可以覆盖默认值：

```
#SBATCH --cpus-per-task=12
#SBATCH --mem=48G
```

- 资源不足时任务会排队，不会超额分配。
- 内存超过申报值时，任务可能以 `OUT_OF_MEMORY` 结束。
- 不确定用量时先用 `debug` 测量峰值。
- 不要为了“保险”申请整机内存；这会阻止其他合法任务启动。
- 公平份额只按 GPU 时间计算，CPU 和内存不增加公平份额用量。

6. 查看任务和历史

```
# 自己的队列
squeue -u "$USER"

# 任务详情
scontrol show job <JobID>

# 预计开始时间（仅供参考）
squeue --start -j <JobID>

# 今天以来的历史
sacct -X -S today -u "$USER" \
  --format=JobID,JobName,Partition,QOS,State,Elapsed,AllocTRES,ExitCode

# 优先级与公平份额
sprio -j <JobID> -l
sshare -l

# 取消自己的任务
scancel <JobID>

# 查看自己的角色与关联
lion-user-show "$USER"
```

统一监控网站可查看各服务器实时 GPU、CPU、内存、队列和近 30 天统计。网站只提供查看功能；提交、取消任务仍在目标服务器命令行完成。

7. 常见排队原因

原因	含义	处理方式
`Resources`	GPU、CPU 或内存暂时不足	等待，或准确缩小资源请求
`Priority`	有更高综合优先级的合法任务	正常等待，不要反复取消重投
`QOSMaxGRESPerUser`	达到个人 GPU 上限	等已有任务结束
`QOSMaxSubmitJobPerUserLimit`	此类任务达到 30 个	取消不再需要的排队任务
`Reservation`	预约未开始、已结束或名称错误	核对管理员提供的预约信息
`ReqNodeNotAvail`	节点维护、故障或被预约	查看原因或联系管理员
`InvalidAccount`	尚未批准 Slurm 权限	联系管理员开通或核对账号

8. GPU 隔离与禁止事项

允许：

- SSH 编辑代码、传文件、查看日志、运行普通 CPU 工具和提交 Slurm 作业。
- 在 Slurm 作业中使用 Conda、venv 和管理员批准的容器方案。
- SSH 下使用无参数 `nvidia-smi` 查看只读 GPU 快照。

禁止：

- SSH 下直接运行 `python train.py`、`nohup`、Jupyter 或 cron 占用 GPU。
- 尝试打开 `/dev/nvidia*`、修改设备权限或绕过 Slurm。
- 使用 Docker `--gpus all`、共享账号或借用账号增加份额。
- 手工修改 `CUDA_VISIBLE_DEVICES` 访问未分配的 GPU。
- 作业结束后故意保留后台 GPU 进程。
- 在提交脚本中手工指定或修改 QoS。

拥有 sudo 权限也不代表可以绕过 Slurm。管理员的 GPU 管理入口有空机保护和审计记录。

9. 跨服务器使用注意事项

在另一台服务器提交前，请检查：

1. 目标服务器是否已经为自己开通 Slurm 关联和正确角色。
2. 代码、数据、Conda 环境和 checkpoint 是否已经同步到目标服务器。
3. 提交脚本中的绝对路径是否适用于目标服务器。
4. 目标服务器的磁盘空间、GPU 型号和健康状态是否满足任务需求。
5. 是否在目标服务器本地执行 `sbatch`，而不是误以为网站会统一提交。

任务不会在服务器之间自动迁移；一台服务器排队不代表另一台也没有空闲 GPU。

10. 任务失败时怎么反馈

先保存 JobID、提交脚本和错误日志，再提供以下输出：

```
scontrol show job <JobID>
sacct -X -j <JobID> \
  --format=JobID,State,Elapsed,AllocTRES,ExitCode
```

如果 `lion-status` 显示 `WARNING` 或 `CRITICAL`，不要自行重启系统或 Slurm 服务；请把状态输出和相关 JobID 一并发给管理员。

11. 常见问题

两张 GPU 能不能自动变成双倍显存？

不能。只有程序实现模型切分、张量并行等机制时，多卡显存才可能共同承载一个模型。

normal 会被博士任务打断吗？

不会。博士采用软优先，只影响后续排队，不抢占正在运行的硕士 `normal` 作业。

为什么博士也只能使用两张 debug GPU？

`debug` 是公共短时通道，所有人统一最多 2 张；博士的 4 卡权益用于 `normal`。

idle 被重新排队后怎么办？

从最近 checkpoint 恢复。若程序不能恢复，应改用 `normal`。

能否直接在 SSH 中启动 Jupyter？

不能直接占用 GPU。需要先获得 Slurm GPU 作业，再在作业内启动 Jupyter；端口转发方式按服务器接入说明执行。

新建 Linux 账号后为什么不能提交？

Linux 账号不会自动获得 Slurm 权限。管理员批准关联后，用户才能提交；默认角色是 `master`。

12. 提交前检查清单

1. 已明确填写 `--gres=gpu:N`。
2. 分区与任务用途一致。
3. 没有填写 `--qos`。
4. GPU 数量确有必要，程序支持多卡。
5. 时间、CPU 和内存接近真实需求。
6. Conda、数据和代码路径在当前服务器有效。
7. `idle` 任务已经验证 checkpoint 保存与恢复。
8. 日志位置可写，关键结果会定期保存。

13. 快速命令表

目的	命令
查看本机健康	`lion-status`
查看只读 GPU 快照	`nvidia-smi`
查看节点	`sinfo`
提交任务	`sbatch job.slurm`
交互调试	`srun -p debug --gres=gpu:1 --time=00:30:00 --pty bash`
查看自己的任务	`squeue -u "$USER"`
查看角色	`lion-user-show "$USER"`
查看当天历史	`sacct -X -S today -u "$USER"`
取消自己的任务	`scancel <JobID>`

> *本手册描述实验室统一默认规则。若某台服务器因硬件、维护或试运行需要存在临时差异，以管理员通知和该服务器实际配置为准。*